Razor 入門

Razor 構文で HTML と C#を組み合わせ、動的な画面を作成する

対応環境　Visual Studio 2022（17.8 以降）／.NET 8.0

Version 1.0

学習内容

変数の表示から始め、Product を 1 件渡す画面、List<Product>を渡す一覧画面、条件による表示の切替を順に作成する。HTML、Razor、Tag Helper、Bootstrap の役割を区別する。

実習の前提

「ASP.NET Core・MVC 入門」で作成した MvcBasicSample を継続使用する。この教材では新しいプロジェクトを作成しない。HelloController と Views/Hello/Index.cshtml が動作していることを前提とする。

【重要】実習のプロジェクト名は MvcBasicSample に統一する。コード中の名前空間も同じ名前を使う。

構成

STEP 1　既存プロジェクトを確認する

STEP 2　変数を表示する

STEP 3　Product を新規作成する

STEP 4　商品 1 件を渡して表示する

STEP 5　商品一覧を渡して表示する

STEP 6　if で表示を切り替える

STEP 7　リンクと表示を整える

共通レイアウトと到達確認

本版の内容

Version 1.0　Visual Studio 2022・.NET 8.0 対応、プロジェクト継続を明記し、1 件と一覧の完成コード、式の区切り、条件分岐の確認手順を整備

STEP 1 既存プロジェクトを確認する

対象ファイル

MvcBasicSample のソリューション／Controllers/HelloController.cs／Views/Hello/Index.cshtml

新規作成か既存編集か

既存ファイルの確認のみ

操作手順

1　Visual Studio 2022 の［プロジェクトやソリューションを開く］から、保存済みの MvcBasicSample の.sln を開く。

2　Controllers/HelloController.cs と Views/Hello/Index.cshtml があることを確認する。

3　F5 で起動し、/Hello/Index を開く。確認後は Shift＋F5 で停止する。

完成コード

この STEP ではファイルを変更しない。開始時点では HelloController の Index()が return View()を実行し、Views/Hello/Index.cshtml には「はじめての MVC」という見出しと本文が記述されている。以降の STEP では、この同じ Controller と View を順に編集する。

コードの意味

Razor は.cshtml に書かれた HTML と C#から、サーバー側で HTML を作る仕組みである。ブラウザに C#の処理そのものが送られるわけではない。

実行確認

「はじめての MVC」が表示されれば開始できる。表示できない場合はプロジェクトを作り直さず、前教材の HelloController と View の場所を確認する。

Razor を読むための基本

記法	意味
@{ … }	C#の処理や変数宣言を書く
@name	変数 name の値を HTML へ出力する
@(name)	出力する C#式の範囲を明示する
@* コメント *@	HTML に出力しない Razor コメント
<!-- コメント -->	HTML に残りソースから読めるコメント

【補足】HTML は見出しや段落などの構造を表す。CSS は色や配置などの見た目を指定し、JavaScript は主にブラウザ内の動作を記述する。

STEP 2 変数を表示する

対象ファイル

Views/Hello/Index.cshtml

新規作成か既存編集か

既存編集。HelloController は変更しない

操作手順

1　Index.cshtml の内容をすべて下のコードに置き換えて保存する。

完成コード

```
@{
    ViewData["Title"] = "あいさつ";  // タブ用のタイトル
    string name = "田中";            // 表示する名前
}

<h1>こんにちは</h1>
<p>@name さん</p>   @* 変数と日本語を空白で区切る *@
<p>@(name)さん</p>  @* 式の終わりを丸括弧で示す *@
```

コードの意味

@{ ... }の中は C#なので、変数宣言にはセミコロンを付ける。ブロック内で宣言した name を、同じ View の後続の HTML で使える。

ViewData は、1 回の画面表示で使用する値をキー名で保存・取得する仕組みである。ここでは Title というキーに「あいさつ」を保存し、共通レイアウトが同じキーから値を取り出してブラウザのタブに表示する。

@name さんは空白で C#の式と日本語を分ける。@(name)さんは丸括弧で式を終わらせるため、表示上の空白を入れずに続けられる。

実行確認

F5 で/Hello/Index を開き、「田中 さん」と「田中さん」の 2 行が表示されることを確認する。

name の値を「鈴木」に変えて保存、停止、再起動すると両方の名前が変わる。

【注意】@name さんと続けて書くと、name さんという 1 つの変数名として解釈され得る。@name さんまたは@(name)さんを使う。@Model.Price の後の「円」も同様に区切る。

【補足】Razor で文字列を出力すると、<や>などは HTML として実行されない安全な文字へ自動変換される。たとえば変数に<b>を入れても、太字を指定する HTML タグではなく文字として表示される。

STEP 3 Product を新規作成する

対象ファイル

Models/Product.cs

新規作成か既存編集か

新規作成

操作手順

1　Models フォルダを右クリックし、［追加］→［クラス］を選ぶ。

2　Product.cs を作成し、内容をすべて下のコードに置き換えて保存する。

完成コード

```
namespace MvcBasicSample.Models;

// 商品 1 件の名前と価格をまとめる
public class Product
{
    public string Name { get; set; } = string.Empty;
    public int Price { get; set; }  // 円単位の価格
}
```

コードの意味

Product は商品 1 件を表すクラスである。Name と Price は値を保持するプロパティで、get は読み取り、set は代入を可能にする。

Name は商品名を保存する string 型のプロパティで、string.Empty は初期値として空文字列を設定する記述である。Price は円単位の価格を保存する int 型のプロパティで、この実習では小数を扱わない。

クラスの定義だけでは商品は作られない。次の STEP で new Product を実行し、値を持つ実体（オブジェクト）を作る。

実行確認

ソリューションをビルドし、エラーが 0 件であることを確認する。まだ Controller から渡していないため、ブラウザは前のあいさつ画面のままでよい。

クラスとオブジェクトの対応

Product クラスで定義する項目	作成するオブジェクトに設定する値の例
Name という string 型のプロパティ	「ハンバーガー」という商品名
Price という int 型のプロパティ	500 という円単位の価格

【重要】ここではデータベースを使用しない。Product は C#のデータをまとめる型であり、このクラスを追加しただけでデータが保存されることはない。

STEP 4 商品 1 件を渡して表示する

対象ファイル

Controllers/HelloController.cs／Views/Hello/Index.cshtml

新規作成か既存編集か

両方とも既存編集

操作手順

1　アプリを停止し、HelloController.cs を下のコードに置き換える。

2　次ページに示す Views/Hello/Index.cshtml の完成コードにも置き換える。Controller が渡す型と View が受け取る型を揃え、2 ファイルを保存してから実行する。

完成コード

Controllers/HelloController.cs

```
using Microsoft.AspNetCore.Mvc;      // MVC の機能を使用
using MvcBasicSample.Models;         // Product を使用

namespace MvcBasicSample.Controllers;

public class HelloController : Controller
{
    public IActionResult Index()
    {
        // 商品 1 件のオブジェクトを作る
        var product = new Product
        {
            Name = "ハンバーガー",  // 商品名
            Price = 500             // 価格
        };

        return View(product);       // 商品 1 件を View へ渡す
    }
}
```

完成コードの続きとして、次ページの View も保存する。

STEP 4 View の完成コードと確認

Views/Hello/Index.cshtml

```
@model MvcBasicSample.Models.Product
@{
    ViewData["Title"] = "商品情報";  // タブ用のタイトル
}

<h1>商品情報</h1>
<p>商品名：@Model.Name</p>    @* 受け取った商品の名前 *@
<p>価格：@Model.Price 円</p>  @* 受け取った商品の価格 *@
```

コードの意味

new Product で商品を 1 件作成し、Name と Price へ値を設定する。return View(product)の引数が、View へ渡す実際のオブジェクトである。

@model MvcBasicSample.Models.Product は、この View が Product 型を受け取る宣言である。小文字の@model は型の宣言、大文字の Model は受け取った実体であり、別の役割を持つ。

@Model.Name と@Model.Price は、受け取った Product のプロパティを読み出して表示する。

実行確認

2 ファイルを保存、ビルド後に F5 で/Hello/Index を開く。「商品情報」「商品名：ハンバーガー」「価格：500 円」が表示されれば成功。

Controller と View の型を揃える

```
Controller
    Product product → return View(product)
                          ↓
View
    @model MvcBasicSample.Models.Product
                          ↓
    @Model.Name / @Model.Price
```

【重要】Product は商品 1 件、List<Product>は商品の一覧で、別の型である。商品 1 件を渡したまま View だけを List<Product>に変更すると、実行時に型が合わないエラーになる。

STEP 5 商品一覧を渡して表示する

対象ファイル

Controllers/HelloController.cs／Views/Hello/Index.cshtml

新規作成か既存編集か

両方とも既存編集

操作手順

1　HelloController.cs を下のコードに置き換え、Product を 2 件含む List<Product>を作る。

2　続けて次ページに示す View の完成コードへ置き換える。Controller と View の両方を保存してから実行する。

完成コード

Controllers/HelloController.cs

```
using Microsoft.AspNetCore.Mvc;          // MVC の機能を使用
using MvcBasicSample.Models;             // Product を使用

namespace MvcBasicSample.Controllers;

public class HelloController : Controller
{
    public IActionResult Index()
    {
        // Product を複数まとめる一覧を作る
        var products = new List<Product>
        {
            new Product
            {
                Name = "ハンバーガー",   // 1 件目の商品名
                Price = 500              // 1 件目の価格
            },
            new Product
            {
                Name = "紅茶",           // 2 件目の商品名
                Price = 450              // 2 件目の価格
            }
        };

        return View(products);           // 商品の一覧を View へ渡す
    }
}
```

STEP 5 View の完成コードと確認

Views/Hello/Index.cshtml

```
@model List<MvcBasicSample.Models.Product>
@{
    ViewData["Title"] = "商品一覧";  // タブ用のタイトル
}

<h1>商品一覧</h1>
<ul>
@* 一覧から商品を 1 件ずつ取り出す *@
@foreach (var product in Model)
{
    <li>@product.Name @product.Price 円</li>
}
</ul>
```

コードの意味

List<Product>は Product 型の値を複数保持する一覧である。1 件だけ入っていても、一覧という型であることは変わらない。

foreach は一覧から順に 1 件取り出して処理する構文である。Model は一覧、ループ内の product はその中の 1 件なので、product.Name や product.Price を読める。

は箇条書き全体、<li>は各項目を表す。2 件のデータから 2 個の<li>が生成される。

実行確認

ビルドして/Hello/Index を開き、ハンバーガー500 円と紅茶 450 円の 2 件が表示されることを確認する。

段階	Controller が渡す型	View の宣言
STEP 4	Product	@model …Product
STEP 5	List<Product>	@model List<…Product>

【重要】Controller が List<Product>を渡す場合、View も@model List<Product>で受け取る。本書では、Controller が渡す型と View が宣言する型を同じ表記に揃えて確認する。

STEP 6 if で表示を切り替える

対象ファイル

Views/Hello/Index.cshtml

新規作成か既存編集か

既存編集。HelloController は STEP 5 のまま

操作手順

1　Index.cshtml 全体を下のコードへ置き換える。商品が 0 件かどうかで表示を分ける。

完成コード

```
@model List<MvcBasicSample.Models.Product>
@{
    ViewData["Title"] = "商品一覧";  // タブ用のタイトル
}

<h1>商品一覧</h1>
@* 件数が 0 なら説明だけを表示する *@
@if (Model.Count == 0)
{
    <p>商品はありません</p>
}
else
{
    <p>@Model.Count 件の商品があります</p>
    <ul>
    @foreach (var product in Model)
    {
        <li>@product.Name @product.Price 円</li>
    }
    </ul>
}
```

コードの意味

Model.Count は一覧の件数である。0 なら「商品はありません」を表示し、それ以外なら件数と商品一覧を表示する。@if で C#の条件分岐を始め、else には@を付けない。

実行確認

最初に通常の 2 件が表示されることを確認する。0 件の場合も確認するときは、HelloController で List<Product>を作る処理から 2 個の new Product の部分だけを一時的に削除し、空の一覧を View へ渡す。

再起動して「商品はありません」と表示されたら、Controller を STEP 5 の完成コードへ戻す。戻した後に 2 件を再確認する。

【注意】空の一覧を渡すことと、何も渡さないことは異なる。return View(products)は残す。Model が null の状態では Count を読み取れない。

STEP 7　リンクと表示を整える

対象ファイル

Views/Hello/Index.cshtml

新規作成か既存編集か

既存編集

操作手順

1　Index.cshtml を下の完成コードに置き換える。HelloController は STEP 5 の 2 件の一覧を使う。

完成コード

```
@model List<MvcBasicSample.Models.Product>
@{
    ViewData["Title"] = "商品一覧";  // タブ用のタイトル
}

<h1>商品一覧</h1>
@* Tag Helper で移動先を指定し Bootstrap で見た目を整える *@
<a asp-controller="Hello" asp-action="Index"
   class="btn btn-primary mb-3">商品一覧を表示</a>

@if (Model.Count == 0)
{
    <p>商品はありません</p>
}
else
{
    <p>@Model.Count 件の商品があります</p>
    <div class="row">
    @foreach (var product in Model)
    {
        @* 狭い画面では 1 列で中程度以上の画面では 2 列にする *@
        <div class="col-12 col-md-6 mb-3">
            <h2 class="h5">@product.Name</h2>
            <p>@product.Price 円</p>
        </div>
    }
    </div>
}
```

STEP 7　コードの意味と実行確認

コードの意味

Tag Helper は HTML に似た記法をサーバー側で処理する機能である。asp-controller="Hello"と asp-action="Index"から、該当 Action へのリンク URL を生成する。

Bootstrap は CSS と JavaScript の部品集である。btn と btn-primary はボタン風の外観、mb-3 は下側の余白、row は行、col-12 と col-md-6 は画面幅に応じた列幅を指定する。

col-12 は 12 分割の全幅、col-md-6 は中程度以上の幅で半分を使う指定である。2 件の要素は狭い画面では縦並び、十分に広い画面では横に 2 件並ぶ。

実行確認

/Hello/Index で青いリンクと 2 件の商品が表示される。リンクをクリックすると同じ商品一覧へ移動する。URL は標準ルートの省略規則によって/Hello と表示される場合もある。

ブラウザの幅を広げたり狭めたりし、商品の並びが 2 列と 1 列に切り替わることを確認する。

記法を役割で見分ける

例	分類	働き
や<p>	HTML	画面の構造
@Model.Name	Razor と C#	C#の値を出力
@foreach や@if	Razor と C#	繰り返しや条件分岐
asp-action="Index"	Tag Helper	Action への URL 生成
class="btn btn-primary"	HTML と Bootstrap	見た目の指定

【補足】このリンクは画面を開くためのもので、商品の登録機能はない。外観をボタンにしても、データを保存する処理が自動的に追加されるわけではない。

共通レイアウトと到達確認

次の 3 ファイルは生成済みのものを参照する。以下は説明用の抜粋であり、ファイル全体をこの短いコードへ置き換えない。

Views/_ViewStart.cshtml

```
@{
    Layout = "_Layout";
}
```

各 View の処理に先立って共通レイアウトを指定する。Layout という設定によって、個別の View を共通の外枠へ組み込む。

Views/Shared/_Layout.cshtml

```
<title>@ViewData["Title"] - MvcBasicSample</title>
@RenderBody()
```

共通の外枠にはナビゲーションや CSS の読み込みがあり、RenderBody()の位置に個別 View の本文が入る。Title を変更しても<h1>の文字は自動では変わらない。

Views/_ViewImports.cshtml

```
@using MvcBasicSample
@using MvcBasicSample.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Views 配下で共通に使う名前空間と Tag Helper を指定する。本書は型の所属が見えるように、@model では MvcBasicSample.Models.Product と省略せずに記述する。

到達確認

- ☐ Razor がサーバーで HTML を生成することを説明できる
- ☐ @name さんと@(name)さんの違いを説明できる
- ☐ @model と Model、および Product と List<Product>を区別できる
- ☐ foreach で商品を 1 件ずつ取り出し、if で 0 件の表示を変えられる
- ☐ Tag Helper による移動先と Bootstrap による外観を区別できる

参照資料

Microsoft Learn　Razor 構文リファレンス

https://learn.microsoft.com/aspnet/core/mvc/views/razor?view=aspnetcore-8.0